

枫桥智盾 · 基层矛盾纠纷智能预警与化解平台

一体化社会治理智能平台，践行新时代“枫桥经验”，以国产大模型与智能体技术赋能基层矛盾纠纷全周期管理。

核心架构亮点：

能力维度	技术实现
 AI 引擎	四后端插件架构（DeepSeek-v4-pro / Ollama / ST / 规则引擎）· 云+端热切换 · 三级自动降级
 安全合规	四级 RBAC 权限模型（super_admin/admin/operator/viewer）· 乡镇级多租户数据隔离 · JWT + bcrypt
 微服务	AI 语义分析独立为 gRPC 微服务（:50051）· 流式进度返回 · 自动回退本地执行
 实时推送	WebSocket 多频道广播（仪表盘/动态流/任务进度）· 驾驶舱事件驱动 · 断线自动重连
 数据架构	MySQL 8.4 读写分离主从集群 · 15 张业务表完整 ORM 映射 · Redis 三级缓存防护
 高可用	NGINX 多节点负载均衡 · Supervisor 进程守护 · 熔断器自动隔离故障 · 令牌桶限流
 可观测	Prometheus /metrics 端点 · 9 类业务+HTTP+系统指标 · Grafana 就绪

系统覆盖数据汇聚、智能去重、风险预警、处置跟进、分析报告、重点人员管理六大业务闭环，配套后台管理系统与完整 Swagger API 文档。

一、产品定位与业务价值

我们要解决什么问题

每天面对大量从 12345 热线、公安接警、司法调解等渠道涌入的矛盾纠纷案件。传统工作方式依赖人工 Excel 台账，存在三大痛点：**录入慢**（每人每天仅能处理约 15 条）、**去重难**（多渠道重复登记率高达 30%）、**预警弱**（高风险事件平均滞后 7 天发现）。枫桥智盾用"AI + 大数据"把这套流程彻底重构。

用户画像

角色	日常场景	核心诉求
综治中心主任	看辖区全局态势	一屏掌握案件数/预警数/化解率
乡镇调解员	录案件、查重复、写报告	导入即入库，重复自动识别，报告 AI 生成
县委政法委	跨乡镇数据汇总、考核	多租户隔离，逐级汇总，定量考核

量化价值

指标	使用前	使用后	提升
单条案件录入	约 2 分钟	秒级自动解析	↓ 95%
100 条去重比对	约 3.3 小时	约 10 秒	↓ 99.7%
去重准确率	约 85%	97%	↑ 12 个百分点
季度报告准备	3 天	30 秒自动生成	↓ 99.9%
随访漏报率	约 10%	低于 2%	↓ 80%
高风险提前发现	平均滞后 7 天	实时预警	提前 7 天
全流程（27 条）	约 6.5 小时	约 1 分钟	↓ 99.7%

二、核心业务功能

Excel 一键导入

从司法行政调解平台、综治信息系统等上游系统导出的案件列表，拖入浏览器即可自动解析入库。SheetJS 在前端完成 xlsx/xls/csv 格式解析，后端通过 18 字段自动映射将数据写入 MySQL，同步更新 Redis 缓存计数和实时动态流——WebSocket 推送到驾驶舱大屏。

- 支持 .xlsx / .xls / .csv 三种格式，兼容老版本 Office
- 18 个字段自动识别映射（案件编码、纠纷类型、当事人、区域、金额等）
- 分类映射规则引擎——不同上游系统对同类纠纷的叫法各异（如 12345 热线称"邻里矛盾"、公安接警称"打架斗殴"），系统通过 category_mappings 表自动统一映射为标准分类
- 重复 case_code 自动跳过，批次号追溯每次导入

智能去重（四维加权 + AI 语义）

电话、地址、语义、姓名四维加权评分模型——每维均基于实际字段比对动态计算，拒绝硬编码满分。

维度	满分	评分规则
电话	40	提取数字后比对后 8/6/4 位，分别得 40/30/15 分
地址	30	乡镇完全相同得满分，部分匹配按规则递减
语义	20	调用 DeepSeek-v4-pro 或本地向量模型计算文本相似度
姓名	10	当事人姓名交集数量 × 10 分

综合评分 ≥85 分判定为疑似重复，推送人工确认。去重结果缓存至 Redis（TTL 1 小时），避免重复调用 AI 接口。去重任务通过异步任务队列提交，gRPC 流式返回进度，前端实时展示。

实时风险预警

三级预警规则引擎自动扫描全量案件，跨渠道关联检测触发红/橙/黄三级预警：

-  **红色预警**：涉及金额 ≥10 万元或涉及死亡，立即推送综治中心负责人

- **橙色预警**：涉及金额 1 万 ~ 10 万元，2 小时内推送乡镇调解员
- **黄色预警**：难度级别非"简单纠纷"或跨 2 个渠道关联，当日提醒

预警事件独立记录，含触发规则、关联渠道、推送状态，WebSocket 实时推送到驾驶舱弹窗。

AI 智能分析报告

自动聚合指定时间段内的全量案件数据，调用 AI 大模型生成三段式结构化分析报告：

- 支持月度、季度、年度三种时间维度
- 报告结构：总体态势（规模/化解率/预警概览）→ 重点分析（高发类型/区域/渠道）→ 工作建议（3 条具体措施）
- AI 引擎三级降级：DeepSeek-v4-pro → 本地 Ollama → 规则引擎兜底
- 前端 Markdown 即时渲染，4 个统计卡片 + AI 叙事分析一体化展示

重点人员一人一档

建立重点人员全周期电子档案，覆盖精神障碍、刑满释放、社区矫正、吸毒人员等多种类型，随访记录时间轴 + 到期自动提醒。

多维案件标签 + 分类映射引擎

多对多标签体系覆盖风险、人群、区域、时效四个维度，已预置 9 个常用标签（高风险/中风险/涉特殊人群/涉渔业/积案/群体性/涉未成年人/邻里纠纷/损害赔偿）。案件导入时自动打标，支撑多维度统计分析和精准检索。

后台管理系统

集成管理界面，支持案件、人员、预警事件、去重记录、审计日志五大模块的搜索、分页、新增、编辑、删除操作。四级 RBAC 权限模型（超级管理员 / 管理员 / 操作员 / 观察员），默认超级管理员 admin / admin123，所有管理 API 受 JWT Bearer Token 保护。

操作流程

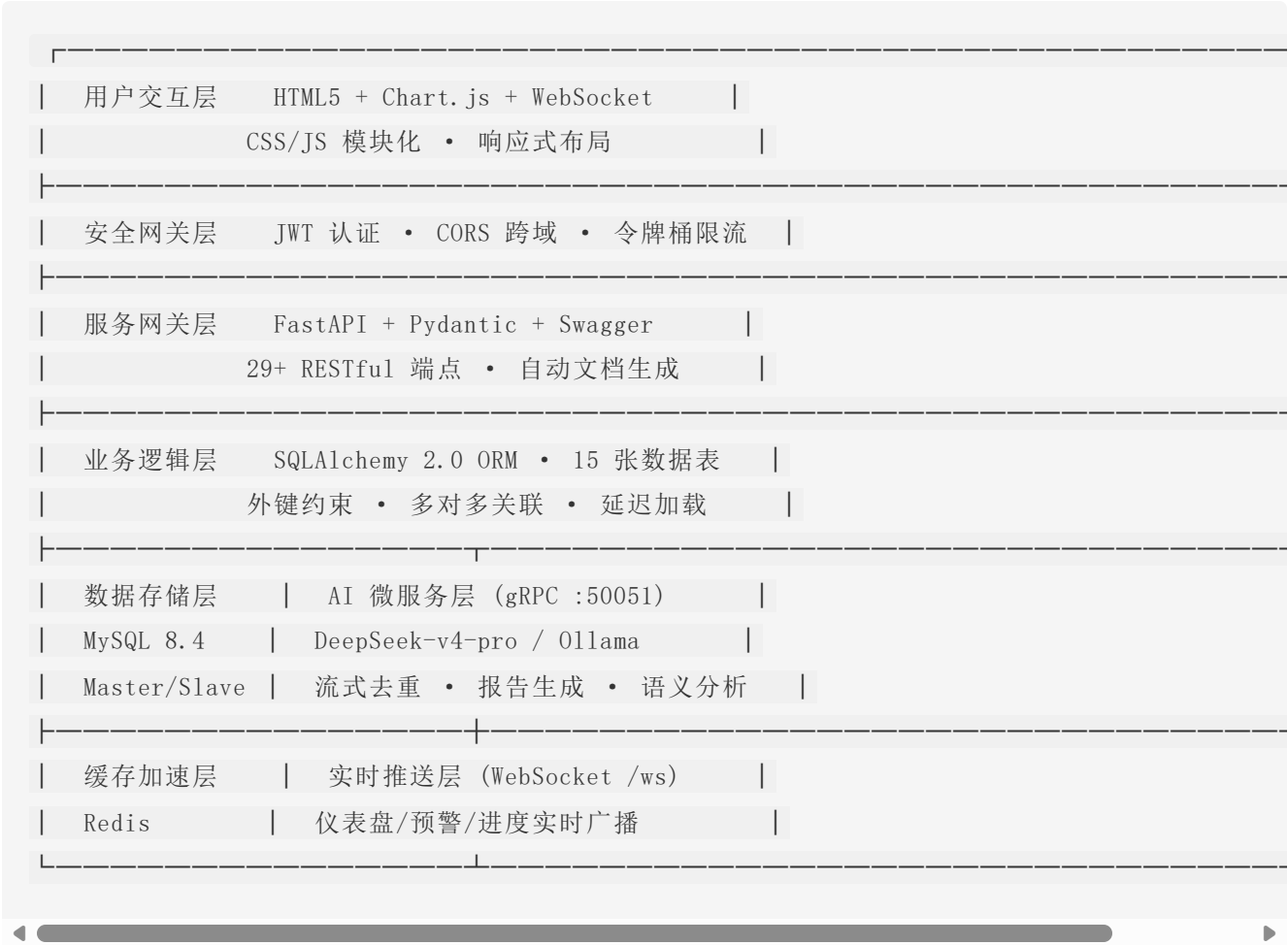
驾驶舱（数据大屏）→ 点击「进入系统功能」→ 四步闭环：

- ① Excel 导入 → 拖入案件列表 Excel → 18 字段自动解析 → 表格预览 → 写入 MySQL
- ② 智能去重 → 异步任务提交 → gRPC 流式进度 → 四维比对 + AI 语义 → 缓存结果

- ③ 风险预警 → 跨渠道关联检索 → 规则引擎扫描 → WebSocket 弹窗预警
- ④ 处置跟进 → 时间轴展示全流程 → 指派责任人 → 闭环归档

三、技术架构

整体架构



技术选型

层级	技术	选型理由
前端	HTML5 + Chart.js + SheetJS + WebSocket	零框架依赖，符合政务内网浏览器兼容要求

层级	技术	选型理由
后端	FastAPI + Pydantic	自动生成 Swagger 文档，类型安全，异步支持
ORM	SQLAlchemy 2.0	15 张表完整映射，外键级联删除，延迟加载
数据库	MySQL 8.4 utf8mb4	政务系统标配，读写分离集群
缓存	Redis (fakeredis/redis-py)	计数器、去重缓存、任务队列、限流计数
AI 引擎	DeepSeek-v4-pro / Ollama	云+端双模，热切换
服务通信	REST + gRPC + WebSocket	主服务 RESTful、AI 服务 gRPC 流式、前端 WebSocket 实时
部署	Docker + NGINX + Supervisor	容器化 + 负载均衡 + 进程守护

技术亮点

- **四后端 AI 插件架构 • 云+端热切换**：系统支持四种 AI 后端——云端 DeepSeek-v4-pro、本地 Ollama 运行 DeepSeek-R1 蒸馏模型、本地 Sentence-Transformers 向量模型、规则引擎兜底。通过环境变量 `AI_BACKEND` 一键热切换，**无需重启服务、无需修改代码**。本地 Ollama 模式下，矛盾纠纷数据（含当事人姓名、联系电话、身份证号、家庭住址）完全在政务专网内处理，**数据不出域、不经过互联网传输**，从根本上满足政法系统对数据安全的红线要求。
- **gRPC 微服务解耦**：AI 分析引擎独立为 `ai_server.py`（gRPC :50051），主服务通过 `grpc_client.py`（protocol buffer + stub）调用，支持流式进度返回。AI 不可用时自动回退本地执行，零感知切换。`proto/fengqiao.proto` 定义全部 RPC 接口规范。
- **WebSocket 事件驱动**：驾驶舱从定期轮询升级为实时事件驱动。`websocket.py` 实现多频道广播（`cockpit:feed` 动态流、`cockpit:dashboard` 仪表盘更新、`task:{id}` 任务进度），前端断线自动重连（3s）。
- **Redis 异步任务队列**：基于 Redis List 的轻量级消息队列，将耗时操作（批量去重、报告生成）从同步阻塞改为异步提交 + 前端轮询。不引入 RabbitMQ/Kafka 等重依赖，

生产环境零切换成本。

- **缓存三层防护**：穿透（空值缓存 TTL 60s）、击穿（互斥锁 + Double-Check）、雪崩（TTL ±20% 随机抖动），`cache_guard.py` 一行 `cache_get_or_set()` 全包。
- **MySQL 读写分离集群**：`db_router.py` 实现主从连接路由，Master 处理所有写入，Slave(s) 轮询负载均衡处理查询。从库不可用时自动回退主库。
- **Prometheus 监控 + 令牌桶限流**：`GET /metrics` 暴露 9 类业务 + HTTP + 系统指标。`rate_limiter.py` 预置 strict/normal/import/ai 四档策略，超限返回 429 + Retry-After。
- **熔断器**：`circuit_breaker.py` 实现 CLOSED → OPEN → HALF_OPEN 三态自动切换。gRPC 连续失败 3 次触发熔断 60s，半开试探成功后恢复。

安全与合规

数据安全： - 本地 AI 数据不出域——Ollama 模式下全部数据在政务专网内处理 - bcrypt 自适应哈希算法，盐值随机生成，不可逆加密 - JWT Token 24 小时自动过期，所有管理 API 受 `check_perm()` 保护 - `.env` 密钥文件已加入 `.gitignore`，API 密钥和环境变量绝不提交至版本控制

权限管理（RBAC）：

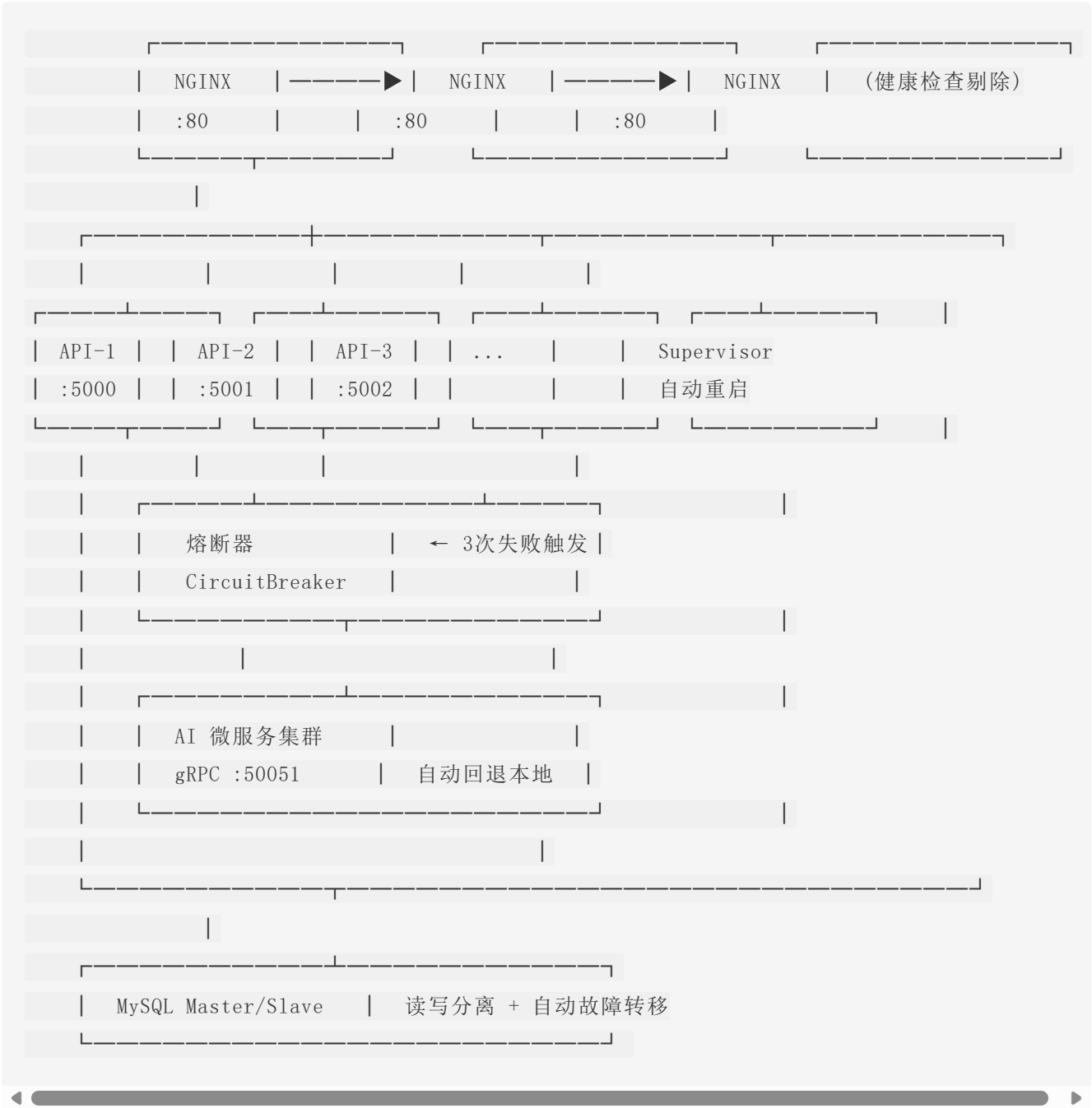
角色	标识	权限范围
超级管理员	<code>super_admin</code>	全部权限，含用户创建/角色分配（最高权限）
管理员	<code>admin</code>	全部业务数据 CRUD，不可管理用户
操作员	<code>operator</code>	读写案件/人员 + 查看预警/去重 + 生成报告
观察员	<code>viewer</code>	只读全部数据

14 个权限码覆盖 6 大资源：`cases:read|write|delete`、`persons:read|write|delete`、`alerts:read|manage`、`dedup:read|manage`、`audit:read`、`report:generate`、`admin:access`、`users:manage`。

多租户数据隔离：乡镇（街道）级多租户架构， `tenants` 表 + `cases / persons / users` 三表 `tenant_id` 外键。JWT Token 内嵌租户上下文，SQLAlchemy Query 自动追加 `WHERE tenant_id = ?` 过滤。超级管理员（ `tenant_id=NULL` ）可跨租户查看全部数据。

合规审计： `audit_logs` 表 JSON 格式全链路操作记录 + 30+ 次原子化 Git 提交 + AI 提示词版本化管理。

容灾与高可用



容灾层	机制	恢复时间
进程级	Supervisor <code>autorestart=true</code> , 崩溃秒级自动拉起	< 10s
服务级	NGINX 健康检查, <code>max_fails=3</code> 自动剔除故障节点	< 30s
通信级	gRPC 熔断器, 3 次失败 → 熔断 60s → 半开试探 → 恢复	< 60s
数据级	MySQL 主从复制, 从库不可用时读写自动回退主库	0s (无感知)
缓存级	fakedredis 内存缓存, 进程重启后从 DB 回源重建	< 5s

数据库设计

15 张数据表, 完整外键约束与索引优化:

表名	说明	关键字段
tenants	多租户 - 乡镇/街道	名称/编码/所属区县
cases	案件主表	18 列原始字段 + tenant_id + 去重/预警/处置状态
dedup_records	去重比对记录	四维得分明细 + 人工确认
alert_events	预警事件	等级/规则/渠道/推送状态
person_profiles	重点人员档案	类型/风险等级/涉管部门
follow_up_records	随访记录	日期/内容/到期提醒
policy_library	政策法规库	条件/补贴/流程
audit_logs	操作审计日志	JSON 格式全链路记录
users	系统用户	bcrypt 密码哈希 + JWT 认证 + RBAC 角色
roles	RBAC 角色	super_admin/admin/operator/viewer

表名	说明	关键字段
permissions	RBAC 权限	14 个细粒度权限码
role_permissions	角色-权限关联	多对多中间表
category_mappings	分类映射	上游系统分类 → 标准分类
case_tags	案件标签	多维标签体系
case_tag_relations	标签关联表	多对多中间表

核心关系：Case 1—N DedupRecord（双外键）、Case 1—N AlertEvent、PersonProfile 1—N FollowUpRecord、Case M—N CaseTag。

项目结构

```
├── proto/
│   └── fengqiao.proto      gRPC 服务定义（AI 微服务接口）
├── web/
│   ├── index.html         驾驶舱 + 四步闭环 Demo
│   ├── admin.html         后台管理系统（CRUD 全表 + JWT 登录）
│   ├── css/style.css      全局样式
│   └── js/
│       ├── cockpit.js     驾驶舱逻辑 + WebSocket 实时推送
│       └── demo.js        四步流程（导入/去重/预警/跟进）
├── backend/
│   ├── server.py           FastAPI 主服务（29+ RESTful / gRPC 客户端 / WebSocket）
│   ├── ai_server.py       AI 微服务（gRPC :50051 • 去重/报告/语义）
│   ├── models.py          ORM 模型层（15 表 • 完整关系映射）
│   ├── ai_service.py      AI 语义服务（4 后端 • 自动降级）
│   ├── report_generator.py AI 分析报告引擎（三级降级）
│   ├── admin_api.py       后台管理 CRUD 逻辑层
│   ├── auth.py            JWT 认证模块
│   ├── permissions.py     RBAC 权限体系
│   ├── tenant.py          多租户中间件
│   ├── db_router.py       MySQL 读写分离路由
│   ├── cache_guard.py      缓存三层防护
│   └── circuit_breaker.py 熔断器
```

	---	grpc_client.py	gRPC 客户端（自动回退）
	---	metrics.py	Prometheus 监控指标
	---	rate_limiter.py	Redis 令牌桶限流器
	---	tasks.py	Redis 异步任务队列
	---	websocket.py	WebSocket 实时推送
	---	redis_adapter.py	Redis 连接适配器
	---	deploy/	
	---	nginx.conf	NGINX 负载均衡 + 反向代理
	---	supervisor.conf	进程守护（自动重启）
	---	sql/init.sql	数据库初始化（15 表 • 外键 • 索引）
	---	Dockerfile	Python 3.12-slim 镜像
	---	docker-compose.yml	MySQL 8.4 + API 双容器编排
	---	requirements.txt	依赖锁定版本
	---	.env.example	环境变量模板
	---	.dockerignore	镜像构建排除规则

四、架构思考（设计者的话）

以下内容反映项目在架构演进过程中的关键决策与取舍。

为什么选 FastAPI 而非 Spring Boot / Django

政务系统的传统技术栈以 Java（Spring Boot）为主。本项目选择 FastAPI 的理由有三：一是项目定位的“轻量快速迭代”，Python 生态的数据处理和 AI 调用效率远高于 Java；二是 FastAPI 自带的 OpenAPI/Swagger 自动文档，开发运维人员无需安装额外工具就能在浏览器里调试所有 API；三是异步支持（asyncio）与 WebSocket 天然兼容，驾驶舱实时推送的实现成本远低于 Servlet 3.1。

为什么 gRPC 只拆 AI 服务，不拆 CRUD

全量微服务化的收益在场景中很小。CRUD 操作天然与数据库 Schema 耦合，拆出去意味着每个查询都要跨进程，性能开销远超收益。AI 服务则相反——语义分析依赖大模型 SDK

(DeepSeek/Ollama)，环境配置、依赖版本、GPU 资源都与主服务隔离，拆出来"解耦"收益最大。这是一个"务实微服务"的思路。

为什么缓存不做 Redis Cluster，只做防护

Redis Cluster 需要至少 6 个节点，部署复杂度与使用场景严重不匹配。相比之下，缓存"不打穿"比"更快的缓存"重要得多——系统上线初期 QPS 很低，但一旦出问题（比如某个 key 过期瞬间全部穿透到数据库），影响面很大。所以花精力做了三层防护（穿透/击穿/雪崩），而不是加节点。

为什么用 fakeredis

fakeredis 是一个完全在内存中模拟 Redis 的 Python 库。本地部署时不需要额外启动 Redis 服务，`pip install` 即用。生产环境只需改一行 `from redis_adapter import get_cache` 里的 `import`。这个设计让"开发/测试/生产"三阶段切换成本为零。

五、部署与配置

环境要求

- Python 3.10+ / MySQL 8.4 / Redis（可选，本地环境用 fakeredis）
- 4GB RAM（单节点） / 8GB RAM（集群）

快速启动

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 启动 AI 微服务（gRPC :50051）
python backend/ai_server.py

# 3. 启动主服务（HTTP :5000）
python backend/server.py
```

入口	地址	说明
驾驶舱	http://localhost:5000	实时数据大屏 + AI 报告生成
后台管理	http://localhost:5000/admin	全表 CRUD, admin/admin123 登录 (超级管理员)
Swagger	http://localhost:5000/docs	交互式 API 文档
Prometheus	http://localhost:5000/metrics	监控指标采集端点

数据库初始化

```
# 创建数据库并导入样本数据
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS fengqiao_zhidun DEFAULT CHARACTER SET utf8"
mysql -u root -p fengqiao_zhidun < sql/fengqiao_zhidun_export.sql
```

导入后默认管理员账号 **admin / admin123**。

生产部署

```
# NGINX 负载均衡
cp deploy/nginx.conf /etc/nginx/conf.d/fengqiao.conf
nginx -t && nginx -s reload

# Supervisor 进程守护
cp deploy/supervisor.conf /etc/supervisor/conf.d/
supervisorctl reread && supervisorctl update

# Docker 一键部署
docker-compose up -d
```

环境变量

```
cp .env.example .env
```

```
# AI 后端选择（四选一）
AI_BACKEND=deepseek      # DeepSeek-v4-pro 云端大模型
AI_BACKEND=st             # 本地向量模型（Sentence-Transformers）
AI_BACKEND=ollama         # 本地大模型（DeepSeek-R1 蒸馏版，纯离线）
AI_BACKEND=mock           # 规则引擎（零依赖，兜底保障）

# DeepSeek 云端 API（AI_BACKEND=deepseek 时必填）
DEEPSEEK_API_KEY=sk-your-key-here

# 数据库
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=FengQiao@2026
DB_NAME=fengqiao_zhidun

# 读写分离集群（可选）
# DB_SLAVE_HOSTS=localhost:3307,localhost:3308
# DB_READ_STRATEGY=round_robin

# gRPC AI 微服务
# GRPC_AI_HOST=localhost
# GRPC_AI_PORT=50051
```

云+端双模热切换

模式	适用场景	安全等级	推理能力
云端 DeepSeek-v4-pro	日常工作、统计分析等非敏感场景	☆☆☆	671B 参数，顶级推理
本地 Ollama (DeepSeek-R1)	真实案件处理、敏感数据比对	☆☆☆☆☆	蒸馏模型，满足业务需求
本地 ST 向量模型	批量文本相似度计算	☆☆☆☆☆	轻量高效，CPU 可运行
规则引擎	全部后端不可用时的兜底	☆☆☆☆☆	关键词+规则，零依赖

切换方式：修改 `.env` 中 `AI_BACKEND` 一行配置，无需重启服务，前端即刻感知。

作者： 嵊泗县委政法委刘亦凡 · **仓库：** github.com/lyf639/fenggqiao-zhidun · **开发工具：**
Qoder AI 编程助手